

SS_Decomp_Exercise

September 8, 2025

```
[1]: import numpy as np
import pandas as pd
from plotnine import *
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression

# Set the seed for reproducibility
np.random.seed(42)

%config InlineBackend.figure_format = 'retina'
```

```
[2]: !pip show plotnine
```

```
zsh:1: /Users/lokeshmuvva/Documents/Temp/ENV/linreg/bin/pip: bad interpreter:
/Users/lokeshmuvva/Documents/regression_f25/Code/linreg/bin/python3: no such
file or directory
Name: plotnine
Version: 0.15.0
Summary: A Grammar of Graphics for Python
Home-page: https://plotnine.readthedocs.io/en/stable
Author:
Author-email: Hassan Kibirige <has2k1@gmail.com>
License: The MIT License (MIT)
```

Copyright (c) 2022 Hassan Kibirige

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR

IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Location: /opt/miniconda3/lib/python3.13/site-packages

Requires: matplotlib, mizani, numpy, pandas, scipy, statsmodels

Required-by:

```
[3]: def generate_data(sigma):
    X = np.linspace(start=1, stop=10, num=10).reshape(-1, 1) # Array w/ 1 column, the -1 means 'infer the # of rows'

    # Define parameters
    beta0 = 4
    beta1 = 3

    # Generate epsilon as a 1D array of  $N(0, \sigma^2)$  RVs
    epsilon = sigma * np.random.randn(10)

    # Generate y using SLR model
    y = beta0 + beta1 * X.flatten() + epsilon

    return X, y
```

```
[4]: def ss_decomp(X, y):

    # Fit a linear regression model
    model = LinearRegression()
    model.fit(X, y)

    # Make predictions
    y_pred = model.predict(X)

    # Calculate the mean of y
    y_mean = np.mean(y)
    n = len(y)

    # Calculate SS quantities
    SST = np.sum((y - y_mean) ** 2).round(4)
    SSR = np.sum((y_pred - y_mean) ** 2).round(4)
    SSE = np.sum((y - y_pred) ** 2).round(4)

    F_stat = (SSR/1)/(SSE/(n-2))
```

```

# Output the sum of squares decomposition
print(f"SST (Total Sum of Squares): {SST}")
print(f"SSR (Regression Sum of Squares): {SSR}")
print(f"SSE (Error Sum of Squares): {SSE}")
print(f"SST = SSR + SSE: {np.isclose(SST, SSR + SSE)}")

print(f"Coefficient of Determination, R^2 is: {np.round(1-SSE/SST, 4)}")

# Make a ggplot
df = pd.DataFrame({
    'X': X.flatten(),
    'y': y,
    'predicted': y_pred,
    'y_mean': y_mean
})

gg1 = (
    ggplot(df, aes(x = 'X', y = 'y')) +
    geom_point() +
    # geom_smooth(method = "lm", formula = "y ~ x", se = False) +
    geom_hline(yintercept=y_mean, linetype='dashed') +
    geom_segment(aes(xend = 'X', yend = 'y_mean'), color = "black") + # SST
    components
    ggtitle(f"R^2 is {np.round(1-SSE/SST, 4)}")
)

gg2 = (
    ggplot(df, aes(x = 'X', y = 'y')) +
    geom_point() +
    geom_smooth(method = "lm", formula = "y ~ x", se = False) +
    geom_hline(yintercept=y_mean, linetype='dashed') +
    geom_segment(aes(xend = 'X', y = 'predicted', yend = 'y_mean'), color = "blue", linetype = "dashed") + # SSR components
    geom_segment(aes(xend = 'X', yend = 'predicted'), color = "red", linetype = "dashed") + # SSE components
    ggtitle(f"R^2 is {np.round(1-SSE/SST, 4)}")
)

display(gg1, gg2) # Print the plots

return gg1, gg2 # Return the gg objects if you want to store the plots into variables

```

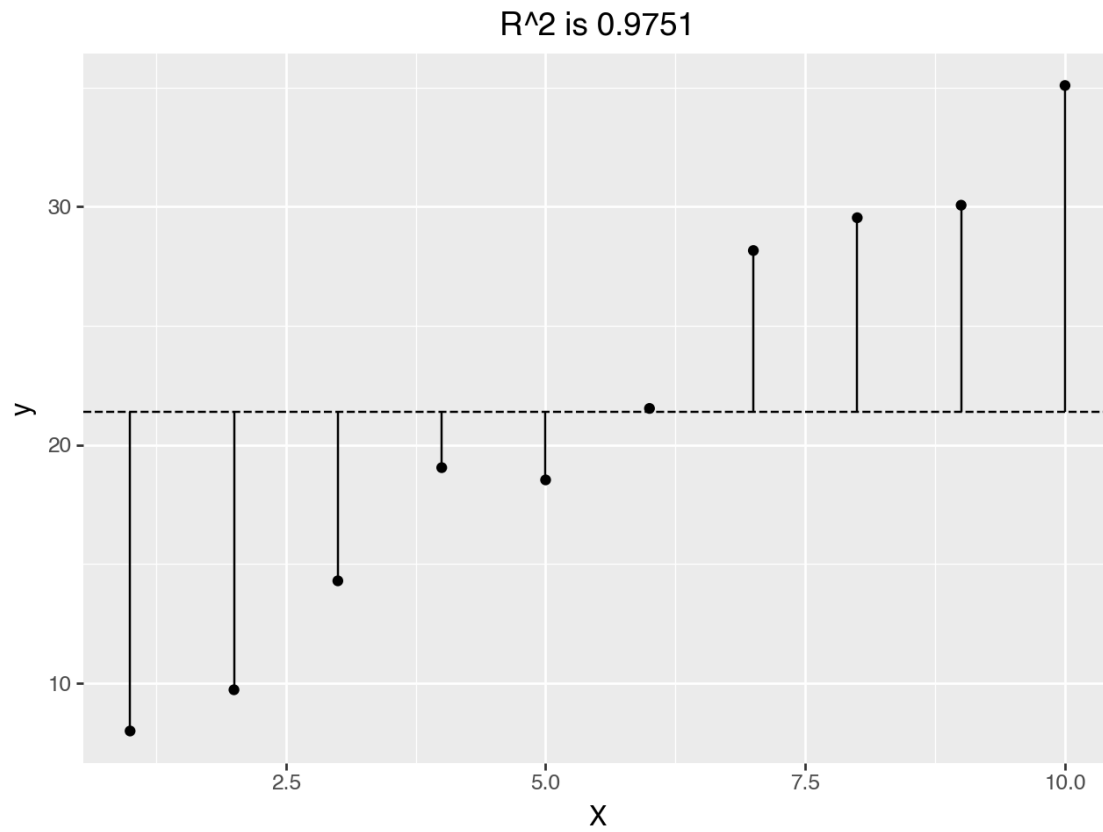
```

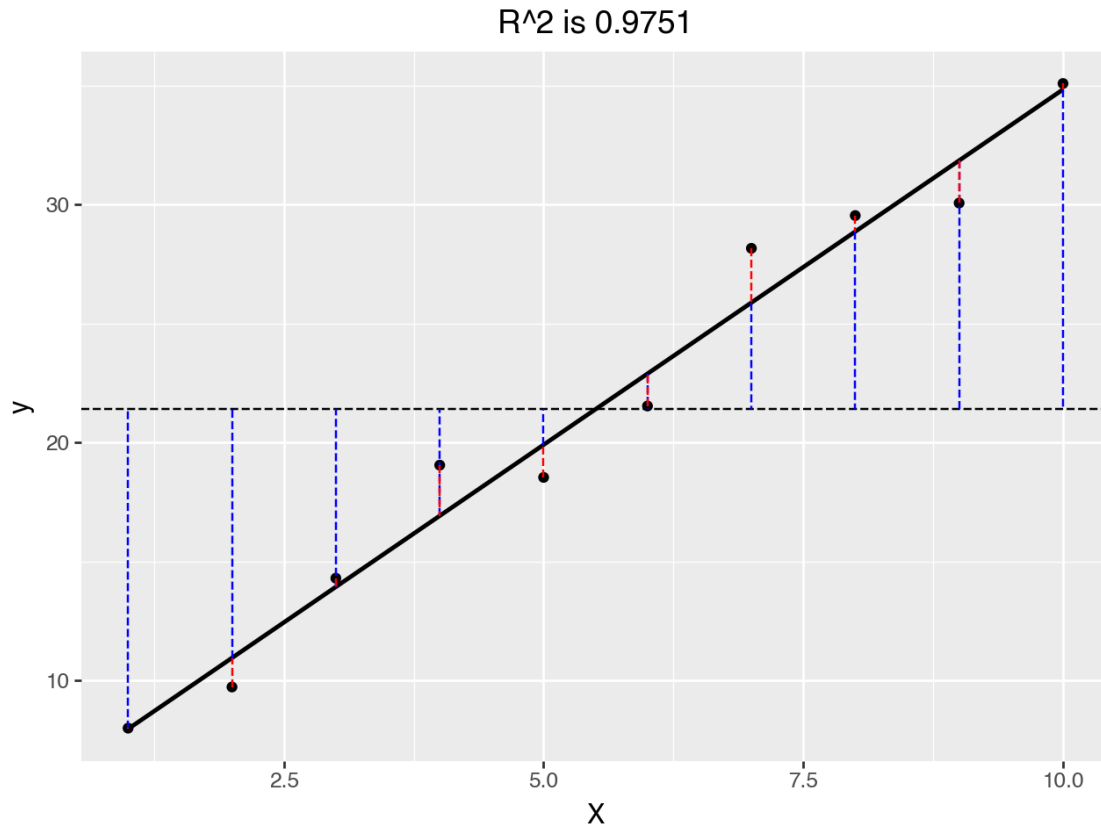
[5]: X, y = generate_data(sigma = 2)
ss_decomp(X, y)

```

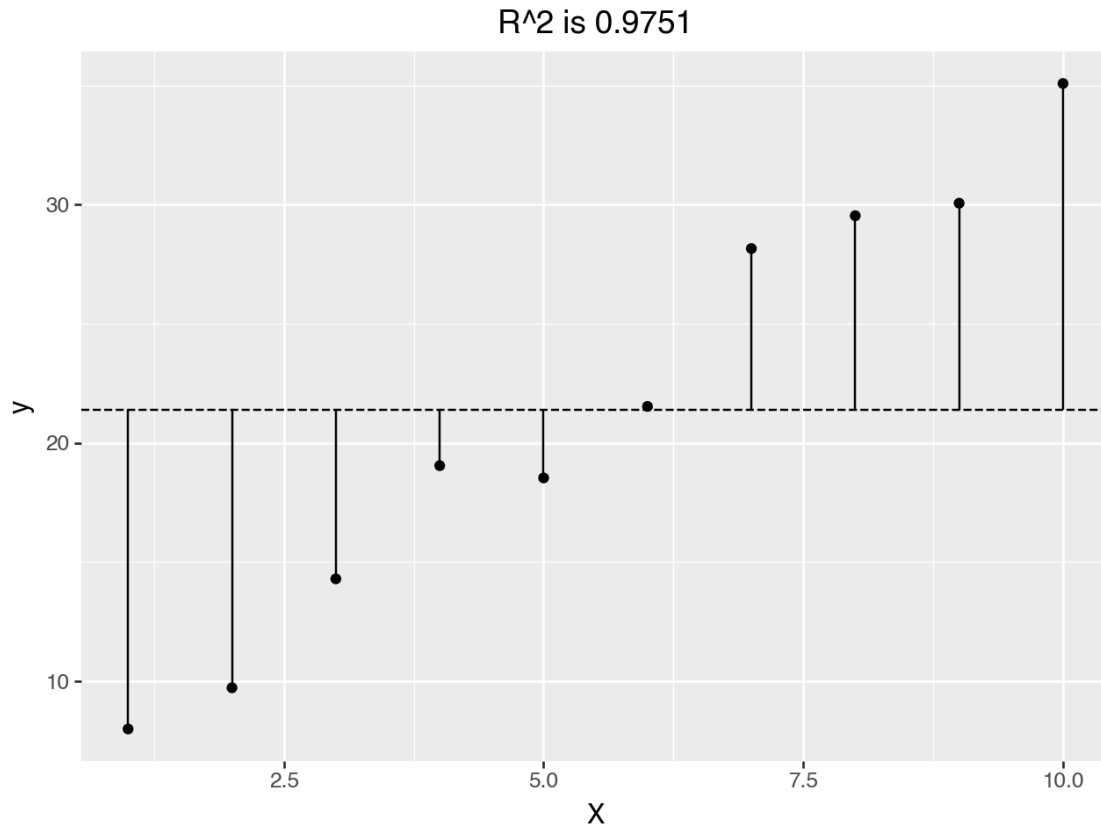
SST (Total Sum of Squares): 754.4873

SSR (Regression Sum of Squares): 735.6843
SSE (Error Sum of Squares): 18.803
SST = SSR + SSE: True
Coefficient of Determination, R^2 is: 0.9751

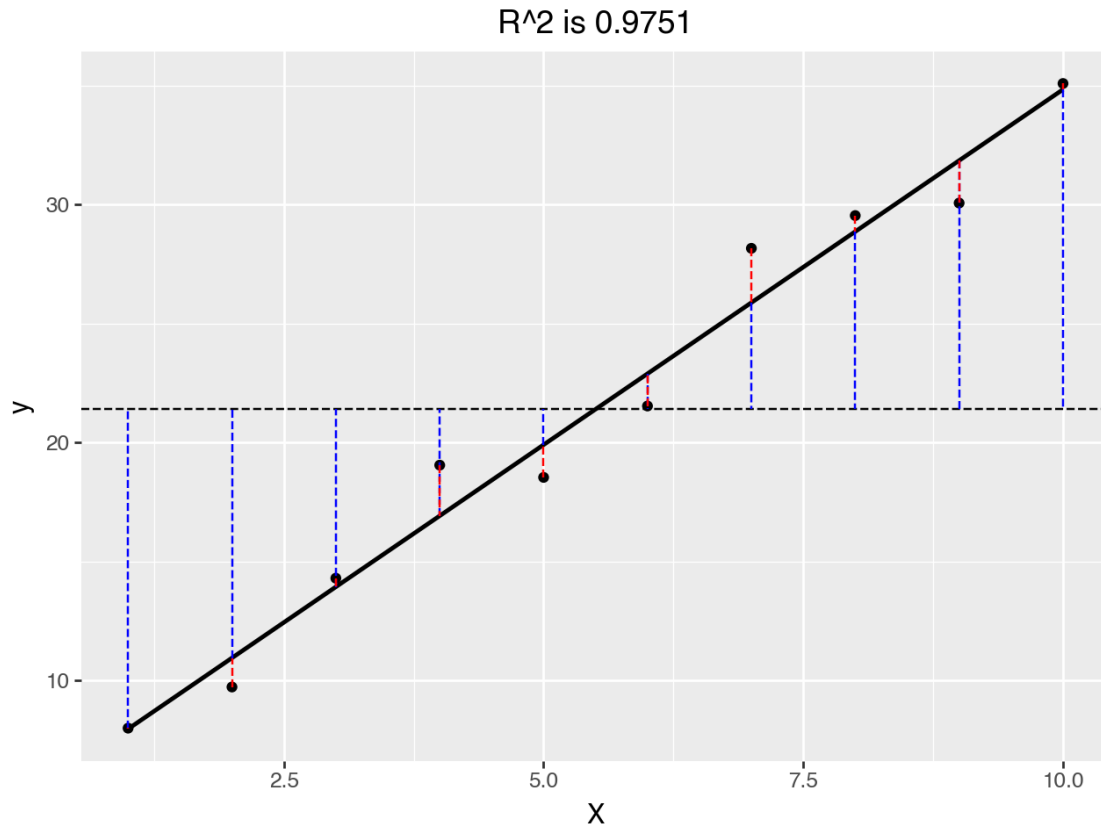




```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```



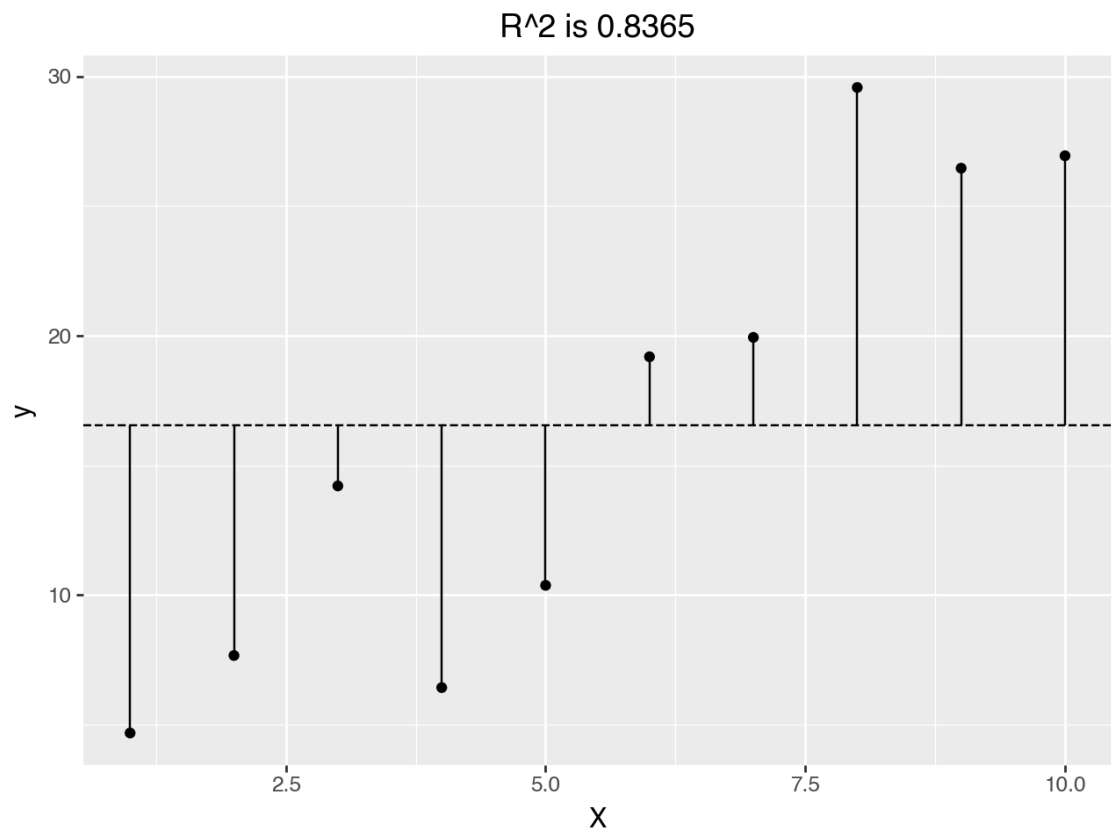
```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```

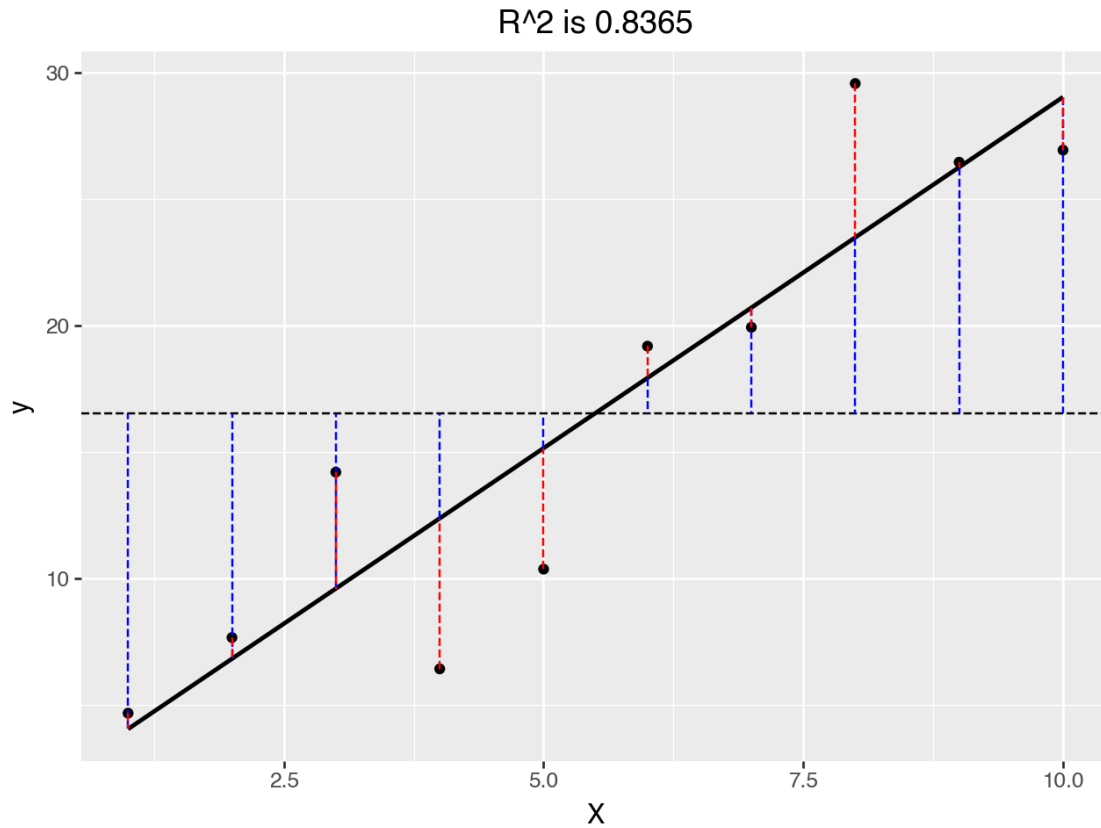


[5]: (<Figure Size: (640 x 480)>, <Figure Size: (640 x 480)>)

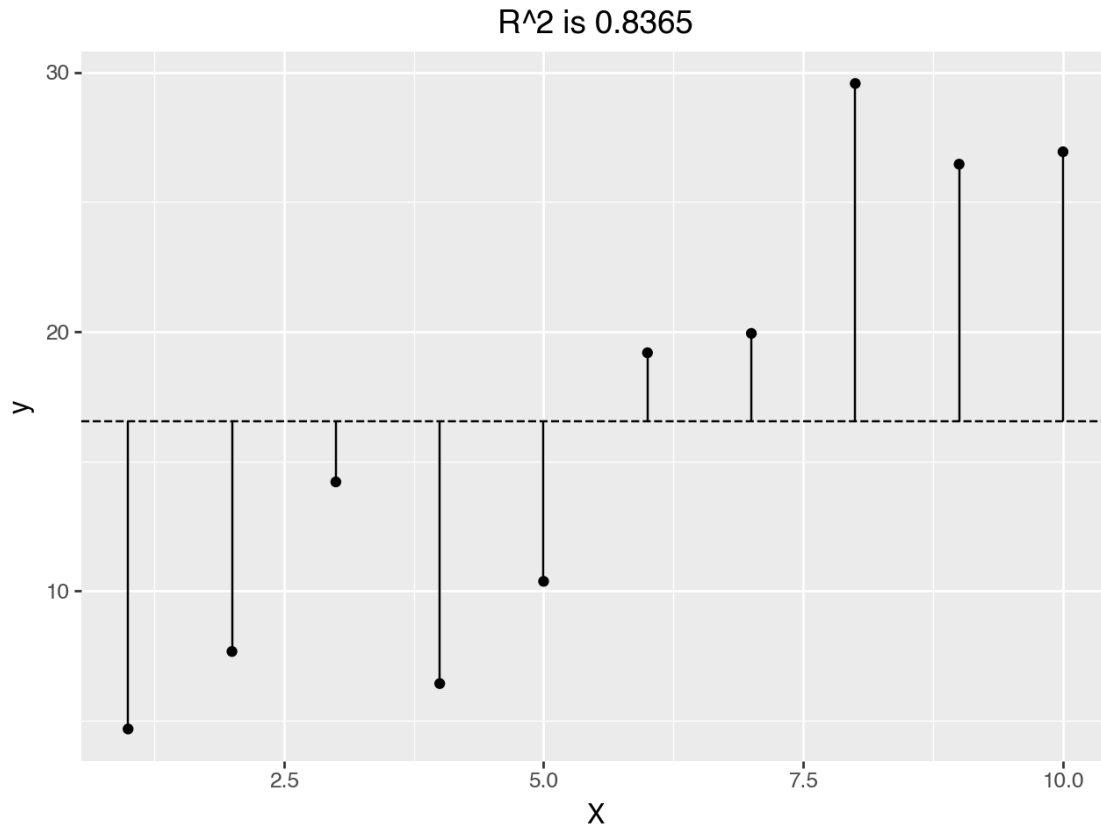
```
[6]: X, y = generate_data(sigma = 5)
      ss_decomp(X, y)
```

```
SST (Total Sum of Squares): 759.7065
SSR (Regression Sum of Squares): 635.5009
SSE (Error Sum of Squares): 124.2056
SST = SSR + SSE: True
Coefficient of Determination,  $R^2$  is: 0.8365
```

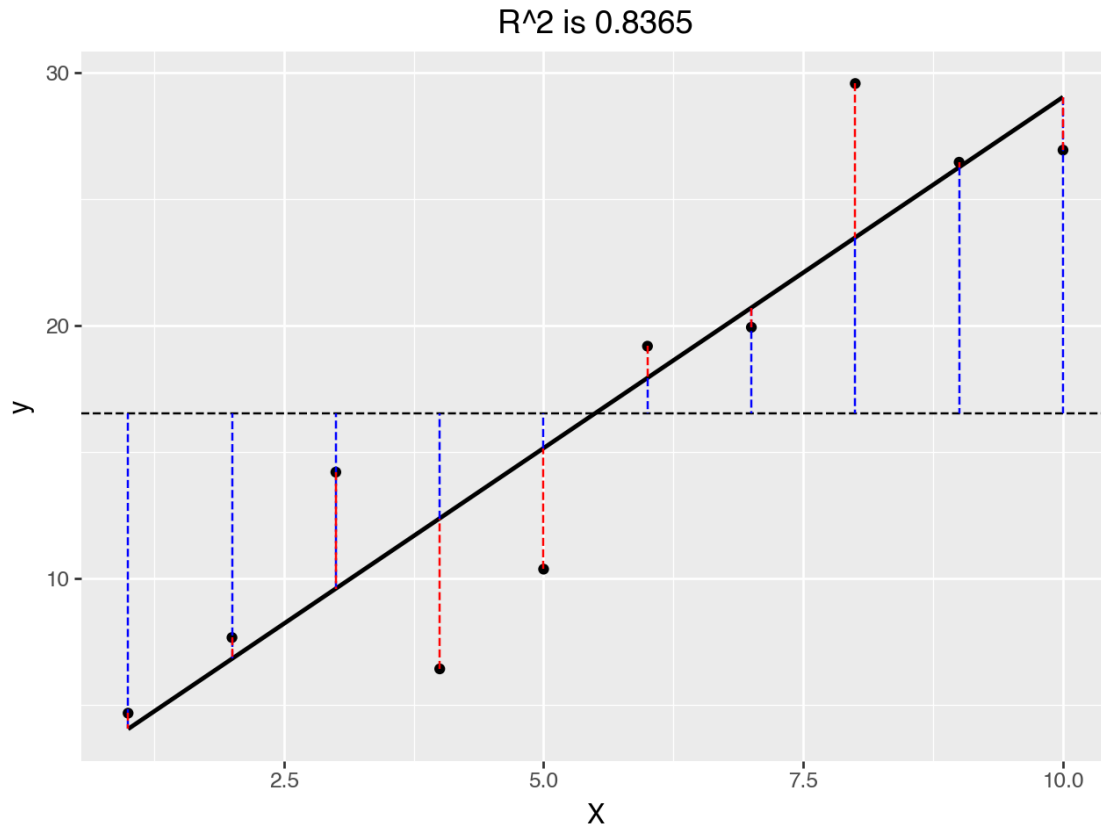




/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and show the plot figure is deprecated and will be removed in a future version. Use plot.show().



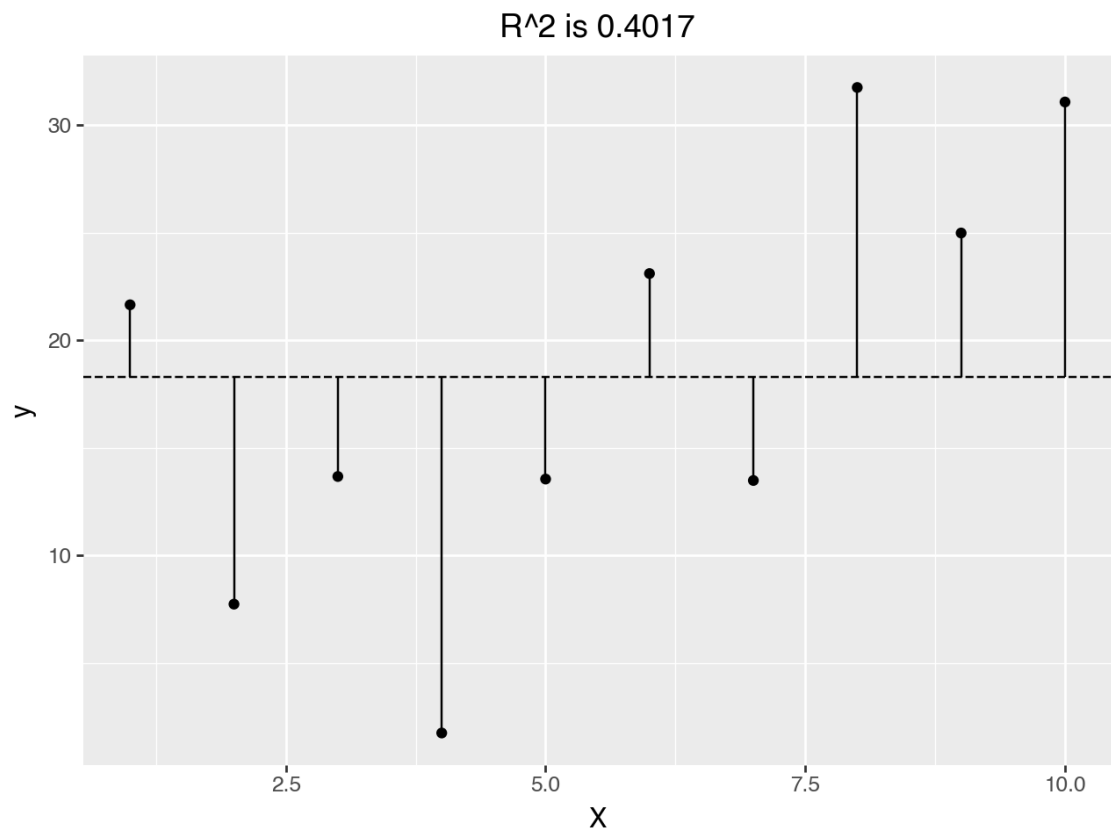
```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```

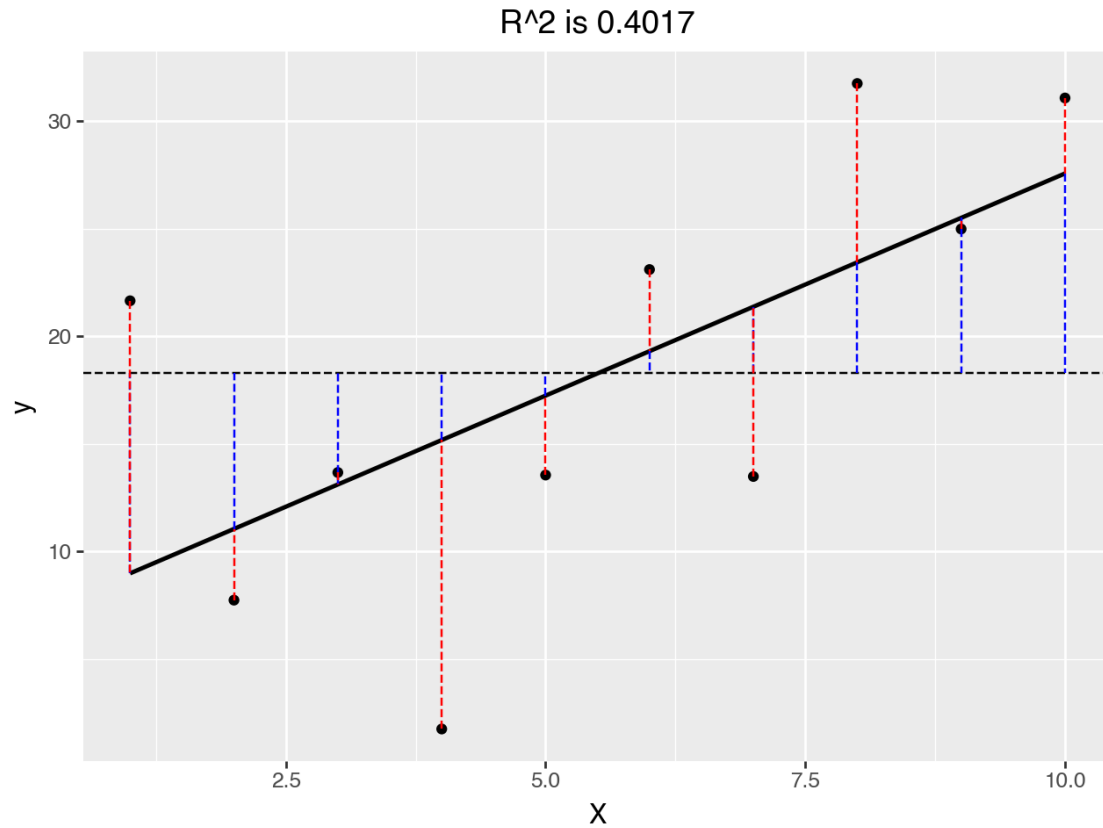


[6]: (<Figure Size: (640 x 480)>, <Figure Size: (640 x 480)>)

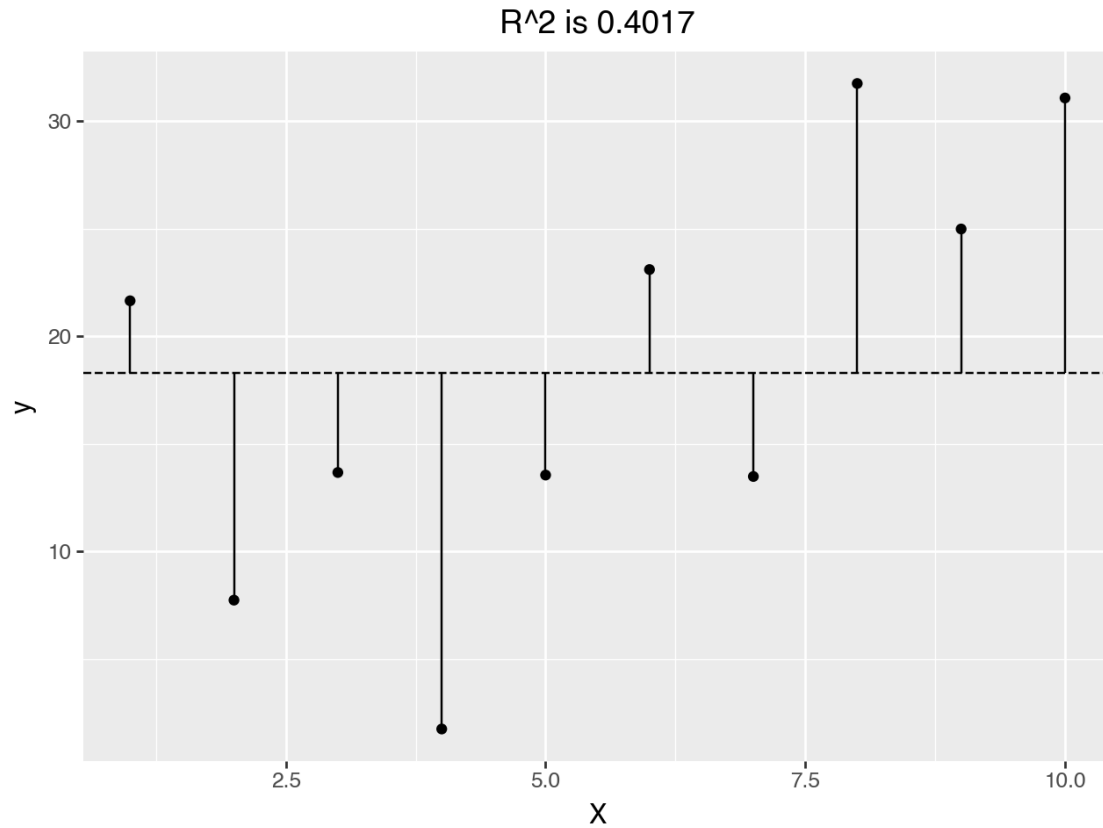
```
[7]: X, y = generate_data(sigma = 10)
      ss_decomp(X, y)
```

```
SST (Total Sum of Squares): 876.0056
SSR (Regression Sum of Squares): 351.8977
SSE (Error Sum of Squares): 524.108
SST = SSR + SSE: True
Coefficient of Determination, R^2 is: 0.4017
```

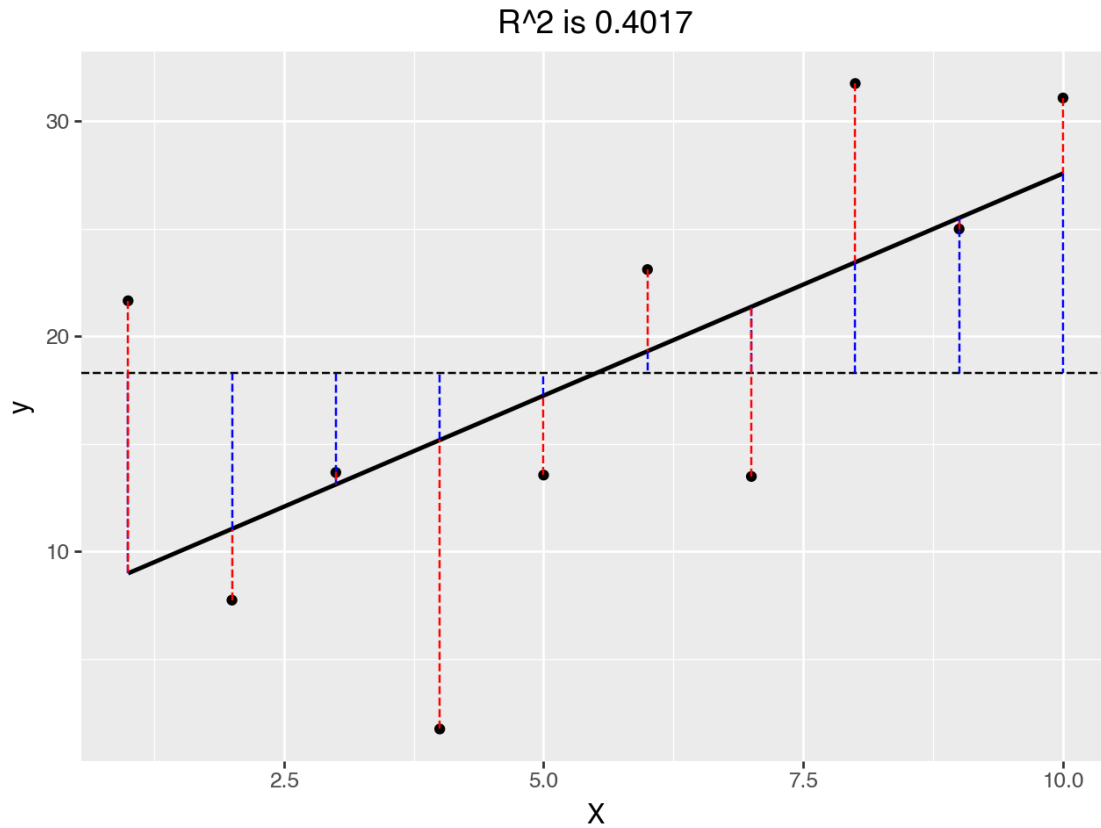




/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and show the plot figure is deprecated and will be removed in a future version. Use plot.show().



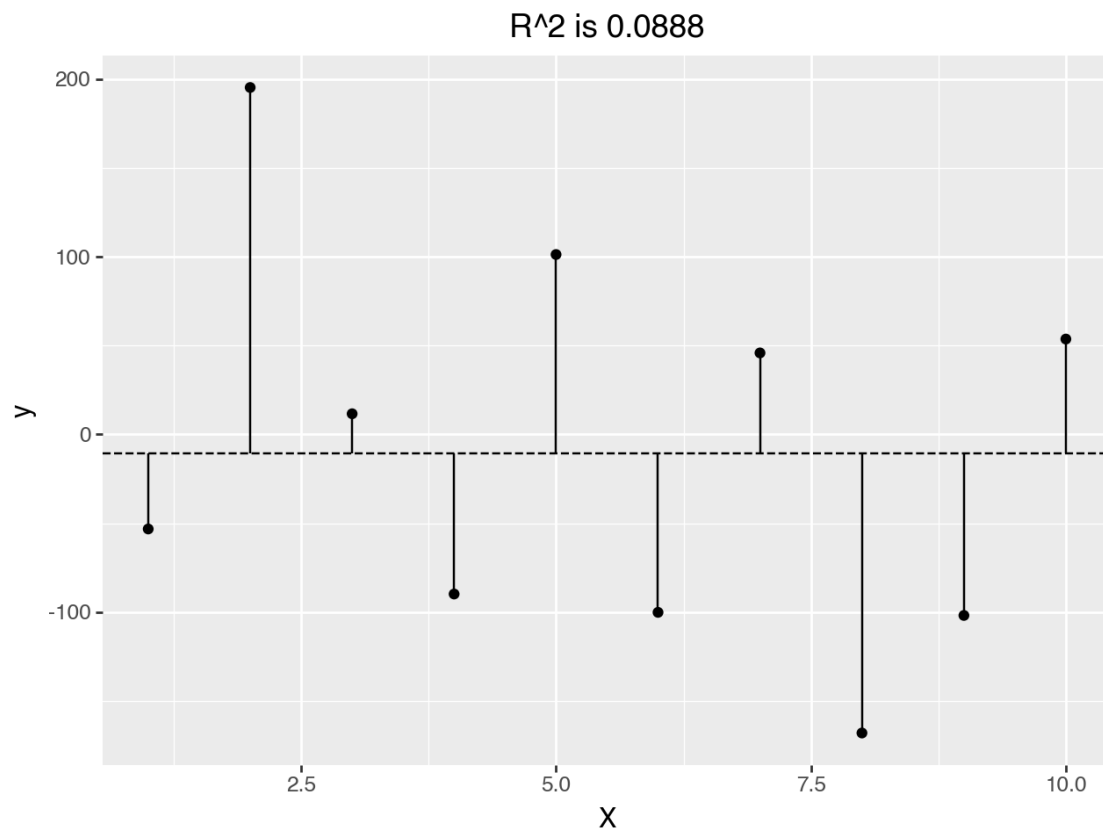
```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```

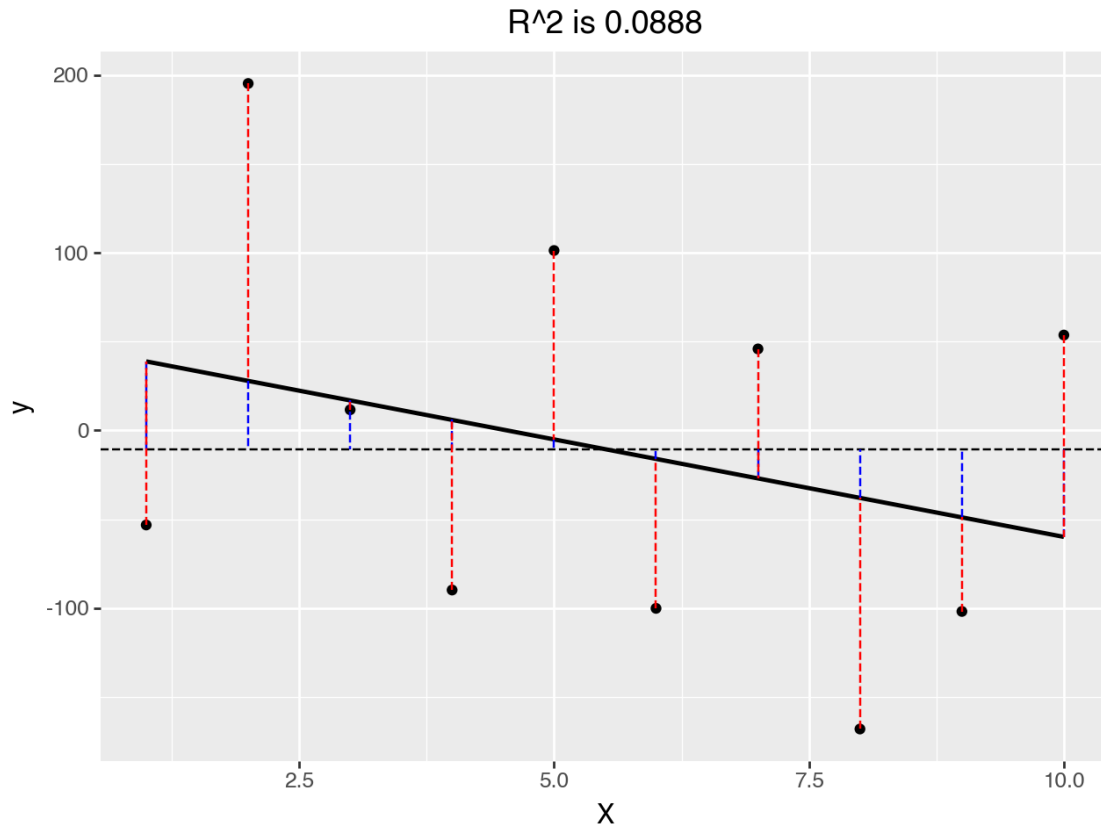


[7]: (<Figure Size: (640 x 480)>, <Figure Size: (640 x 480)>)

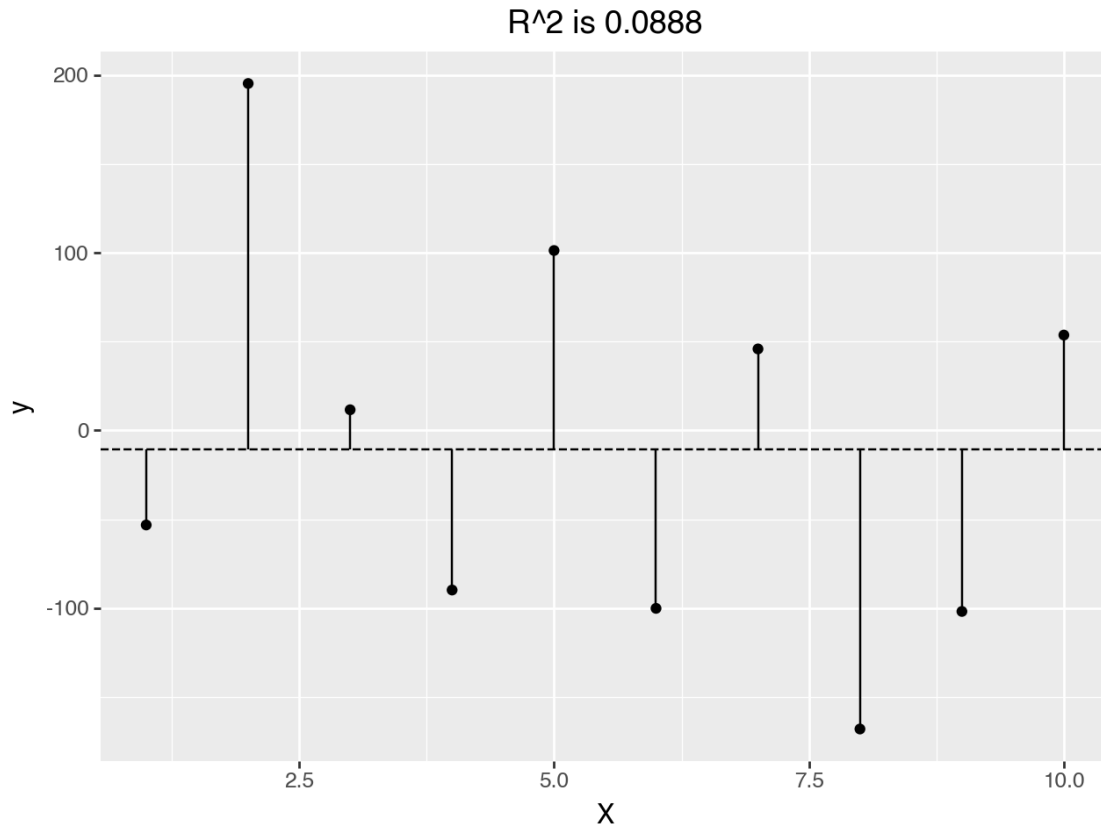
```
[8]: X, y = generate_data(sigma = 100)
      ss_decomp(X, y)
```

SST (Total Sum of Squares): 111867.9128
 SSR (Regression Sum of Squares): 9928.3091
 SSE (Error Sum of Squares): 101939.6037
 SST = SSR + SSE: True
 Coefficient of Determination, R² is: 0.0888

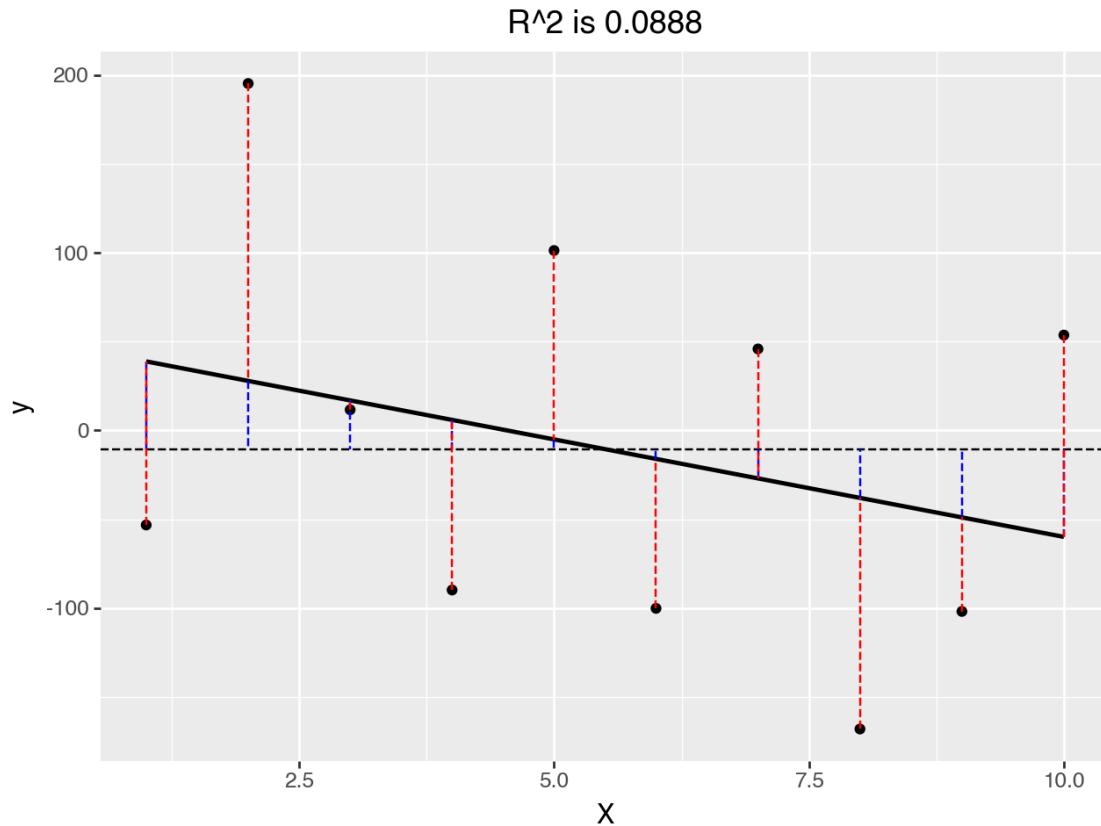




```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```



```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```



[8]: (<Figure Size: (640 x 480)>, <Figure Size: (640 x 480)>)

0.1 Exercises:

1. Modify the data generation function to take the slope, intercept, and sample size as arguments.
2. Modify the SS decomposition function to conduct the F-test for testing whether the true slope is zero or not.
3. Holding the intercept, sample size, and error variance constant, compare how different values of the slope affect:
 - a. the SS decomposition,
 - b. the plots,
 - c. the coefficient of determination, and
 - d. the hypothesis test results.
 What do you find?
4. Holding the slope, sample size, and error variance constant, compare how different values of the intercept affect:

- a. the SS decomposition,
 - b. the plots,
 - c. the coefficient of determination, and
 - d. the hypothesis test results.
What do you find?
5. Holding the intercept, slope, and error variance constant, compare how different values of the sample size affect:
- a. the SS decomposition,
 - b. the plots,
 - c. the coefficient of determination, and
 - d. the hypothesis test results.
What do you find?
6. Write a function which calculates the t-statistic for testing whether the true slope is zero or not. For each simulation you run, calculate both the t-stat and the F-stat. Repeat this $B = 100$ times, then plot the ordered pairs $\{(t_b, F_b)\}_{b=1}^B$ as a scatter plot. What relationship do you observe between them? Can you prove this relation?

```
[9]: def generate_data(sigma, beta0, beta1, n):
    X = np.linspace(start = 1, stop = n, num = n).reshape(-1, 1) # Array w/ 1 column, the -1 means 'infer the # of rows'

    # Define parameters
    # beta0 = intercept
    # beta1 = slope

    # Generate epsilon as a 1D array of  $N(0, \sigma^2)$  RVs
    epsilon = sigma * np.random.randn(n)

    # Generate y using SLR model
    y = beta0 + beta1 * X.flatten() + epsilon

    return X, y
```

```
[10]: def ss_decomp(X, y):

    # Fit a linear regression model
    model = LinearRegression()
```

```

model.fit(X, y)

# Make predictions
y_pred = model.predict(X)

# Calculate the mean of y
y_mean = np.mean(y)
n = len(y)

# Calculate SS quantities
SST = np.sum((y - y_mean) ** 2).round(4)
SSR = np.sum((y_pred - y_mean) ** 2).round(4)
SSE = np.sum((y - y_pred) ** 2).round(4)

# Output the sum of squares decomposition
print(f"SST (Total Sum of Squares): {SST}")
print(f"SSR (Regression Sum of Squares): {SSR}")
print(f"SSE (Error Sum of Squares): {SSE}")
print(f"SST = SSR + SSE: {np.isclose(SST, SSR + SSE)}")

print(f"Coefficient of Determination, R^2 is: {np.round(1-SSE/SST, 4)}")

# Calculate F-statistic
F_stat = (SSR/1)/(SSE/(n-2))

# Calculate F-value
from scipy.stats import f
alpha = 0.05
df1 = 1
df2 = n - 2
F_crit = f.ppf(1 - alpha, df1, df2)

print(f"The F-Statistic is: {F_stat}")
print(f"The Critical F-Value (alpha = 0.05) is: {F_crit}")

if F_stat > F_crit:
    print("We reject the null hypothesis that the slope = 0 . X is a ↵
    ↵significant predictor of Y")
else:
    print("We fail to reject the null hypothesis. X is not a significant ↵
    ↵predictor of Y")

# Make a ggplot
df = pd.DataFrame({
    'X': X.flatten(),
    'y': y,
    'predicted': y_pred,

```

```

        'y_mean': y_mean
    })

    gg1 = (
        ggplot(df, aes(x = 'X', y = 'y')) +
        geom_point() +
        # geom_smooth(method = "lm", formula = "y ~ x", se = False) +
        geom_hline(yintercept=y_mean, linetype='dashed') +
        geom_segment(aes(xend = 'X', yend = 'y_mean'), color = "black") + # SST
    ↪components
        ggtitle(f"R^2 is {np.round(1-SSE/SST, 4)}")
    )

    gg2 = (
        ggplot(df, aes(x = 'X', y = 'y')) +
        geom_point() +
        geom_smooth(method = "lm", formula = "y ~ x", se = False) +
        geom_hline(yintercept=y_mean, linetype='dashed') +
        geom_segment(aes(xend = 'X', y = 'predicted', yend = 'y_mean'), color = "
    ↪blue", linetype = "dashed") + # SSR components
        geom_segment(aes(xend = 'X', yend = 'predicted'), color = "red",
    ↪linetype = "dashed") + # SSE components
        ggtitle(f"R^2 is {np.round(1-SSE/SST, 4)}")
    )

    display(gg1, gg2) # Print the plots

    return gg1, gg2 # Return the gg objects if you want to store the plots into
    ↪variables

```

```

[11]: X, y = generate_data(sigma = 2, beta0 = 2, beta1 = 2, n= 2)
      ss_decomp(X, y)

```

```

/var/folders/8s/b3g9yw6125n1scdnf7737s_c0000gn/T/ipykernel_2692/1426564541.py:28
: RuntimeWarning: invalid value encountered in scalar divide

```

SST (Total Sum of Squares): 0.3748

SSR (Regression Sum of Squares): 0.3748

SSE (Error Sum of Squares): 0.0

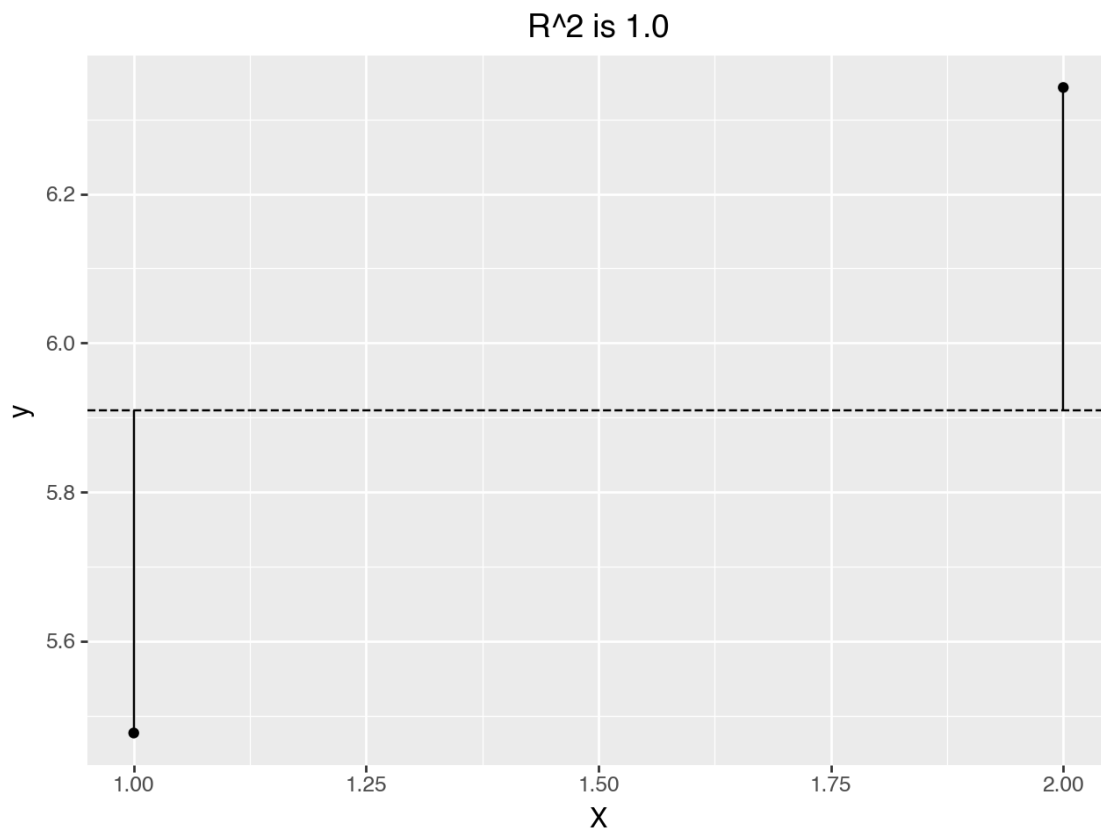
SST = SSR + SSE: True

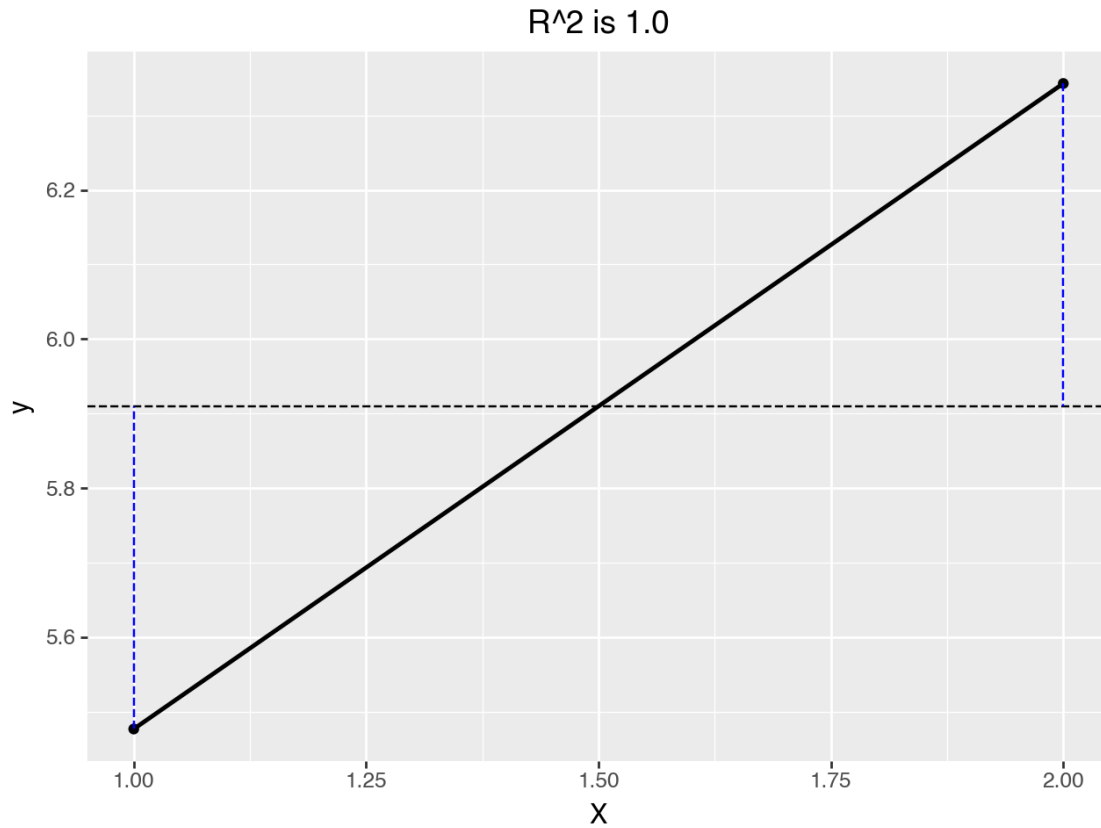
Coefficient of Determination, R² is: 1.0

The F-Statistic is: nan

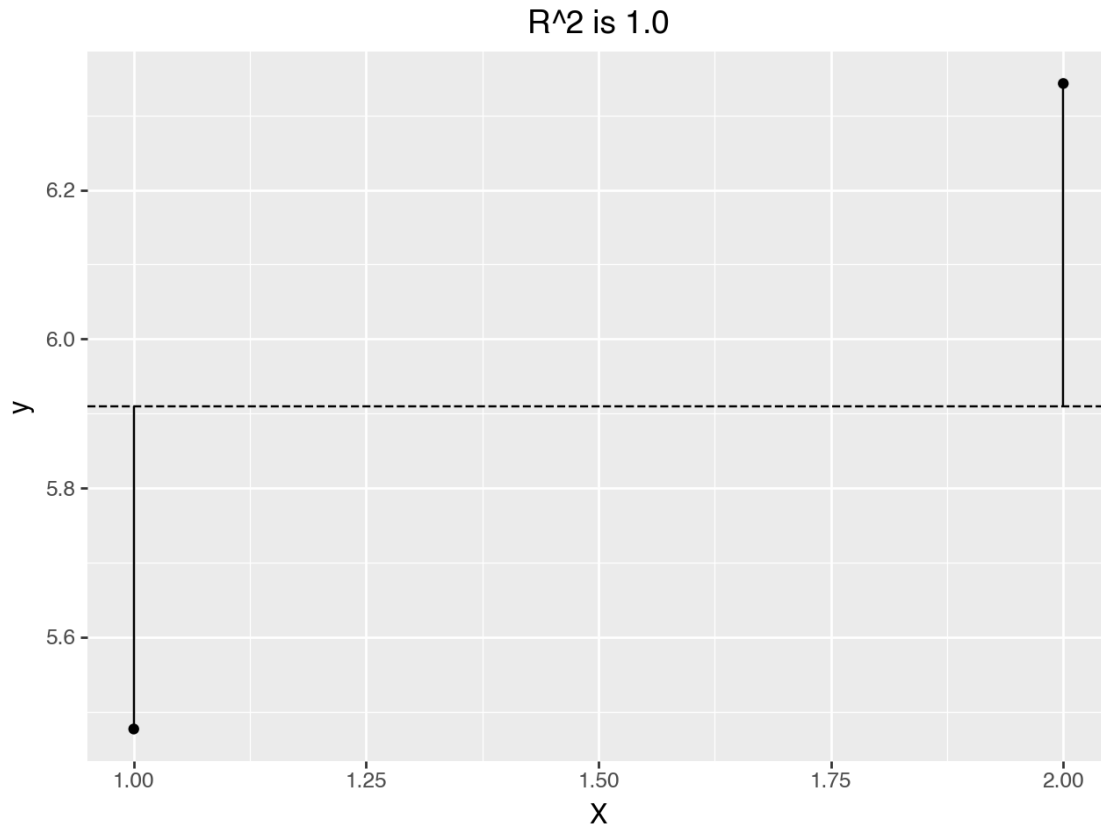
The Critical F-Value (alpha = 0.05) is: nan

We fail to reject the null hypothesis. X is not a significant predictor of Y

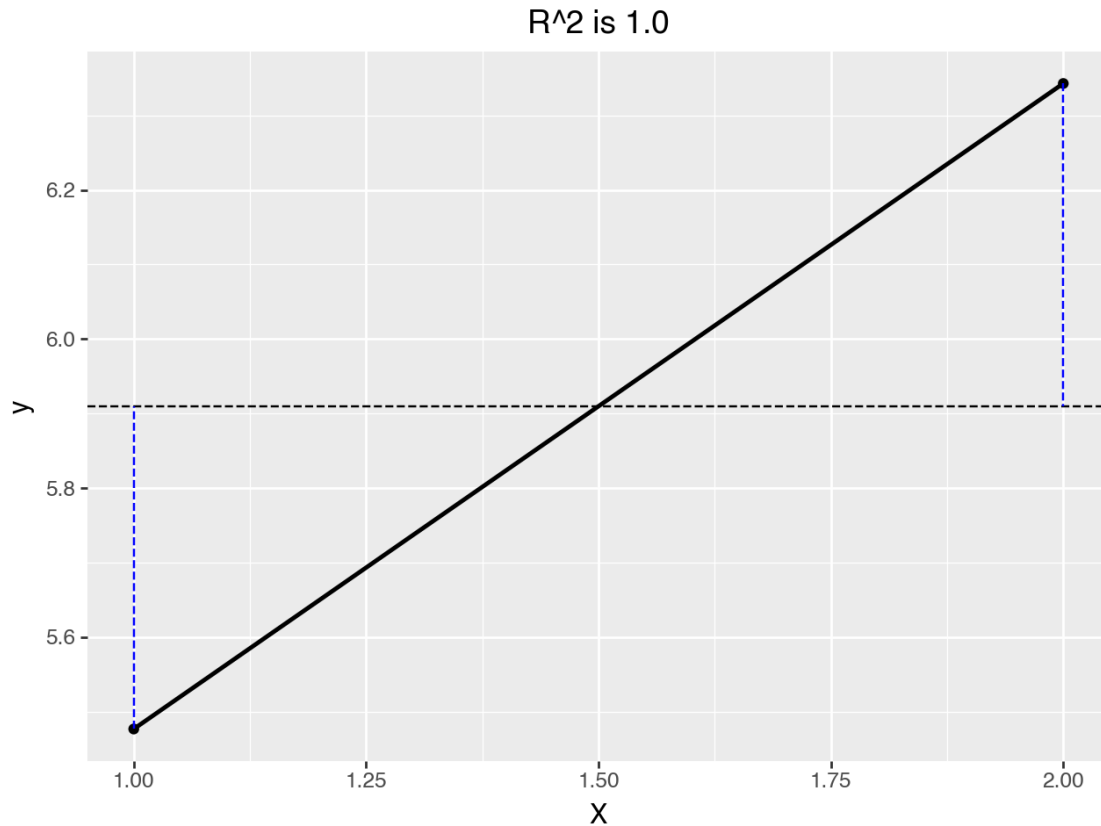




```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```

```
/Users/lokeshmuvva/Documents/Temp/ENV/linreg/lib/python3.9/site-  
packages/IPython/lib/pretty.py:778: FutureWarning: Using repr(plot) to draw and  
show the plot figure is deprecated and will be removed in a future version. Use  
plot.show().
```



```
[11]: (<Figure Size: (640 x 480)>, <Figure Size: (640 x 480)>)
```

```
[12]: def calculate_f_and_t_statistic(X, y):  
  
    # Fit a linear regression model  
    model = LinearRegression()  
    model.fit(X, y)  
  
    # Make predictions  
    y_pred = model.predict(X)  
  
    # Calculate the mean of y  
    y_mean = np.mean(y)  
    n = len(y)  
  
    # Calculate SS quantities  
    SST = np.sum((y - y_mean) ** 2).round(4)  
    SSR = np.sum((y_pred - y_mean) ** 2).round(4)  
    SSE = np.sum((y - y_pred) ** 2).round(4)
```

```

# Output the sum of squares decomposition
print(f"SST (Total Sum of Squares): {SST}")
print(f"SSR (Regression Sum of Squares): {SSR}")
print(f"SSE (Error Sum of Squares): {SSE}")
print(f"SST = SSR + SSE: {np.isclose(SST, SSR + SSE)}")

print(f"Coefficient of Determination, R^2 is: {np.round(1-SSE/SST, 4)}")

# Calculate F-statistic
F_stat = (SSR/1)/(SSE/(n-2))

# Calculate F-value
from scipy.stats import f
alpha = 0.05
df1 = 1
df2 = n - 2
F_crit = f.ppf(1 - alpha, df1, df2)

print(f"The F-Statistic is: {F_stat}")
print(f"The Critical F-Value (alpha = 0.05) is: {F_crit}")

if F_stat > F_crit:
    print("We reject the null hypothesis that the slope = 0 . X is a ↵
↵significant predictor of Y")
else:
    print("We fail to reject the null hypothesis. X is not a significant ↵
↵predictor of Y")

# Calculate t-statistic for slope coefficient
# Get the slope coefficient
slope = model.coef_[0]

# Calculate standard error of slope
MSE = SSE / (n - 2)
X_centered = X - np.mean(X)
SE_slope = np.sqrt(MSE / np.sum(X_centered ** 2))

# Calculate t-statistic for testing H0: slope = 0
t_stat = slope / SE_slope

print(f"The t-Statistic for slope is: {t_stat}")

# Make a ggplot
df = pd.DataFrame({
    'X': X.flatten(),
    'y': y,
    'predicted': y_pred,

```

```

        'y_mean': y_mean
    })

    return t_stat, F_stat

```

```

[13]: # Simulation with B = 100
B = 100
t_stats = []
f_stats = []

for b in range(B):
    # Generate random data
    n = 50 # sample size
    X = np.random.randn(n, 1)
    # Generate y with some relationship to X plus noise
    true_slope = np.random.randn() # Random true slope
    y = true_slope * X.flatten() + np.random.randn(n)

    # Calculate statistics (suppress output for simulation)
    import sys
    from io import StringIO

    # Capture output to avoid printing during simulation
    old_stdout = sys.stdout
    sys.stdout = StringIO()

    t_stat, f_stat = calculate_f_and_t_statistic(X, y)

    # Restore output
    sys.stdout = old_stdout

    t_stats.append(t_stat)
    f_stats.append(f_stat)

# Convert to numpy arrays
t_stats = np.array(t_stats)
f_stats = np.array(f_stats)

# Create scatter plot
plt.figure(figsize=(10, 8))
plt.scatter(t_stats, f_stats, alpha=0.7, s=50)
plt.xlabel('t-statistic', fontsize=12)
plt.ylabel('F-statistic', fontsize=12)
plt.title('Relationship between t-statistic and F-statistic\n(B = 100, ↪simulations)', fontsize=14)
plt.grid(True, alpha=0.3)

```

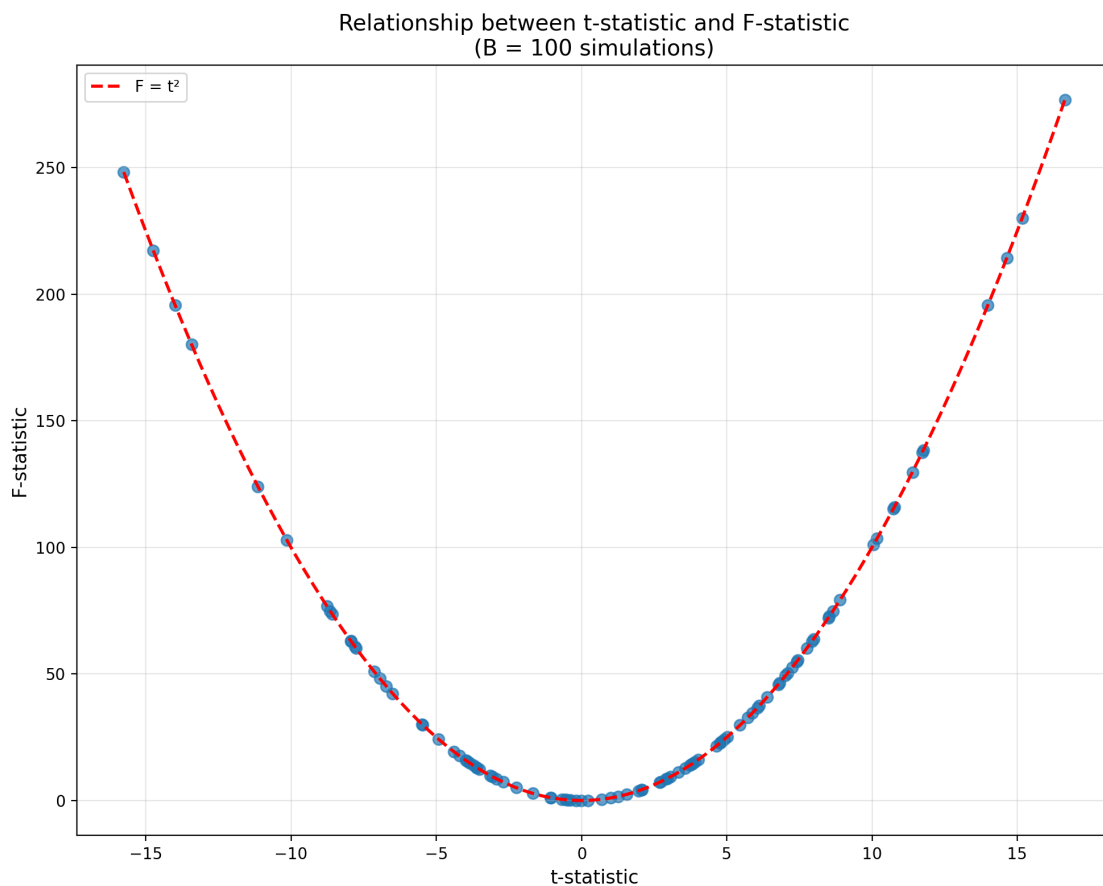
```

# Add the theoretical relationship line:  $F = t^2$ 
t_range = np.linspace(min(t_stats), max(t_stats), 100)
plt.plot(t_range, t_range**2, 'r--', linewidth=2, label='F = t2')
plt.legend()

plt.tight_layout()
plt.show()

# Show some example pairs
print(f"\nFirst 10 (t, F, t2) pairs:")
for i in range(10):
    print(f"t={t_stats[i]:.4f}, F={f_stats[i]:.4f}, t2={t_stats[i]**2:.4f}")

```



First 10 (t, F, t²) pairs:

t=-4.1972, F=17.6163, t²=17.6162

t=-8.6518, F=74.8530, t²=74.8530

t=3.7811, F=14.2971, t²=14.2970

t=-4.3995, F=19.3558, t²=19.3557

t=-8.5848, F=73.6989, $t^2=73.6989$
t=-5.4727, F=29.9506, $t^2=29.9506$
t=7.2441, F=52.4775, $t^2=52.4775$
t=-0.3913, F=0.1531, $t^2=0.1531$
t=3.3370, F=11.1355, $t^2=11.1355$
t=6.3961, F=40.9104, $t^2=40.9104$